

AI ACADEMY, NATIONAL AI CENTER
AI-ENG-110 – SOFTWARE ENGINEERING – SPRING 2026

Topic 4 – Async Research Assistant

Team Binary

Orkhan Aliyev • Habil Huseynov

Final Project Defense – July 2026

`github.com/mrorxan/research-assistant`

The problem in one sentence

What the system does

Ask a research question; we fetch Wikipedia, arXiv, and a web search **in parallel**, then an LLM writes a short answer where **every claim carries a [N] citation** back to a real source.

In

- ▶ One question, ≤ 500 chars
- ▶ Optional -sources
wiki,arxiv

Out

- ▶ Cited answer + references
- ▶ Timing, cache state, failures

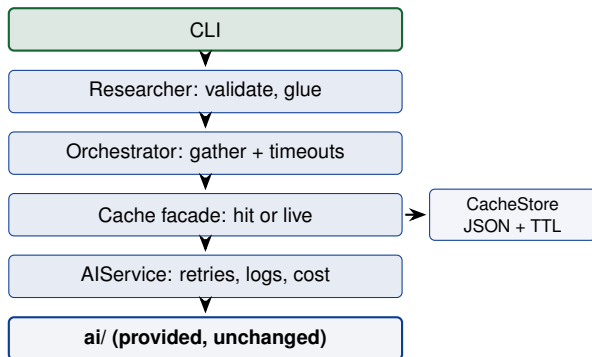
Why

- ▶ LLMs alone hallucinate
- ▶ Grounding in sources = RAG

```
python -m researcher ask "..."
```

Gemini 2.5 Flash Lite + Tavily, swappable by env var

Architecture: one job per layer



Three hard boundaries

- ▶ Only **one module** imports `ai/`
- ▶ Only the facade touches storage
- ▶ Only the orchestrator spawns tasks

Consequence

Swapping the LLM provider = one env var, zero code changes.

Three design decisions we can defend

1. **asyncio, not threads.**

Three I/O-bound HTTP calls per question. One event loop, no GIL contention, and the provided fetchers are already coroutines.

2. **A timeout per task, and one for the LLM too.**

Each source gets its own `asyncio.timeout`, so a slow Wikipedia cannot cancel a fast arXiv. The synthesis call has its own 60 s bound.

3. **Filesystem JSON cache, and never cache empty results.**

Entries are idempotent within their TTL, so no database needed. An empty result means a bad moment; remembering it for 24 h would serve nothing after the source recovers.

Wrapping the AI module: the service layer

Policy	Where it lives
Retries: 3 attempts, backoff 1–8 s, only on <code>ProviderError</code>	<code>AIService</code>
Programmer errors (<code>ValueError</code>) are not retried	<code>AIService</code>
Cache by (source, canonicalised query), TTL 24 h	Cache facade + store
Cost record per LLM call, <code>cost-report</code> command	<code>CostMeter</code>
Structured key=value logs, never a secret	everywhere

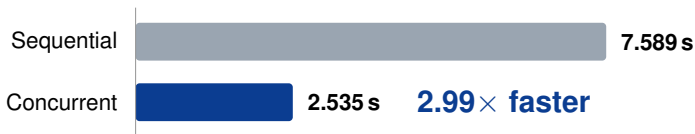
Learned the hard way, live

Wikipedia returns **HTTP 403** unless the User-Agent identifies you with a contact. One shared `httpx` client now carries a policy-compliant agent for all three fetchers.

Handover point: this layer tested, the second half composes it into an app.

Concurrency: the benchmark

Workload: 5 asks $\times$ 3 stub sources, 0.5 s sleep each. Deterministic, so it measures *our orchestrator*, not the network.



- ▶ Theoretical maximum with 3 equal sources is 3 $\times$: **the fan-out costs nothing.**
- ▶ Live runs: 3.6–5.1 s per question, about half of it the LLM call.
- ▶ New bottleneck: the slowest source + synthesis, not fetch parallelism.

Reproduce without any API key: `python scripts/bench.py -N 5 -max-parallel 3`

Robustness: the failure found us

Live incident, July 20. Gemini returned 503 twice during a demo ask.

```
13:36:49 WARNING Retrying in 1 seconds ... 503 UNAVAILABLE
13:36:52 WARNING Retrying in 2 seconds ... 503 UNAVAILABLE
13:36:56 INFO      synthesize_done n_citations=2
```

The user saw a slower answer, not a crash.

The same discipline on every path

- ▶ One source fails or times out: the other two still answer, the output says so.
- ▶ All sources fail: clean error, exit code 3. Bad input: exit code 2.
- ▶ Windows console crashed on a Unicode arrow from the LLM: CLI now forces UTF-8.
- ▶ Bonus: FailoverLLM chains a second provider when configured; chaos-tested.

Sample run (real output)

```
$ python -m researcher ask "What is photosynthesis and what are its main stages?"

A: Photosynthesis is a system of biological processes where
    photopigment-bearing organisms convert light energy into
    chemical energy [1]. [...] The main stages are the
    light-dependent reactions and the Calvin cycle [5, 6].

References:
    [1] (wikipedia) Photosynthesis -- en.wikipedia.org/wiki/Photosynthesis
    [5] (web) Photosynthesis -- education.nationalgeographic.org/...
    [6] (web) The process of photosynthesis -- monash.edu/...

timings: 9210 ms | cache: wikipedia=miss, arxiv=miss, web=miss
```

- ▶ Second identical ask: arxiv=hit, web=hit, 4.8s.
- ▶ All 5 sample questions answered live; records in artefacts/demo_run.json.
- ▶ Estimated cost per question: about \$0.0002.

Testing and quality gates

Numbers

- ▶ **92 tests**: 76 ours + 16 provided smoke
- ▶ Coverage on researcher/: **96 %**
- ▶ mypy: 0 issues ruff: 0 findings
- ▶ All offline; `ai.*` boundary is mocked
- ▶ Same 92 pass **inside the Docker image**

Tests worth naming

- ▶ Timeout isolates one source, others deliver
- ▶ `ValueError` is never retried
- ▶ Empty results are not cached
- ▶ Failover chaos test: primary dies, secondary answers

CI on every pull request

ruff + mypy + pytest with a 60 % coverage gate + smoke tests + Docker build.

Limitations & what we would do next

Honest weak spots

- ▶ Wikipedia title search often returns nothing for long questions; we degrade gracefully.
- ▶ No streaming: the user waits silently during synthesis.
- ▶ Cache has no cross-process lock (harmless at CLI scale, documented).
- ▶ Token counts are estimates; the fixed ai/ interface hides real usage.

Given another week

1. Query rewriting for Wikipedia recall.
2. Streaming synthesis output to the terminal.
3. Failover on by default with a second funded provider.

Team contributions: first half / second half

Orkhan – foundation & service layer

- ▶ Project scaffold, config, models
- ▶ Cache store + cost telemetry
- ▶ AIService: retries, timeouts, logs
- ▶ Cache facade + their test files
- ▶ Report §1, 2, 4, 5; slides 1–5

Habil – composition & shipping

- ▶ Orchestrator, Researcher flow, CLI
- ▶ FailoverLLM, demo + bench scripts
- ▶ Dockerfile, CI workflow, docs
- ▶ Their test files, live demo runs
- ▶ Report §3, 6, 7, 8; slides 6–10

The handover

Orkhan built and tested everything up to the service layer; Habil composed it into the concurrent app and shipped it. Commits split accordingly, roughly 50/50.

Questions?

Team Binary

`github.com/mrorxan/research-assistant`

Pick any file in the repo; we will walk through it.